Q1

```
n = int(input("Enter a number:"))
if(n%2==0):
  print("Even")
else:
  print("Odd")
```

```
Enter a number:5
Odd
```

```
a = int(input("Enter a number:"))
b = int(input("Enter a number:"))
c = int(input("Enter a number:"))

if(a>b):
  if(a>c):
    print("A is greater")
  else:
    print("C is greater")
elif(b>c):
  if(b>a):
    print("B is greater")
  else:
    print("C is greater")
else:
  print("C is greater")
```

```
Enter a number:10
Enter a number:20
Enter a number:30
C is greater
```

```
num1 = int(input("Enter a number:"))
op = input("Enter an operator:")
num2 = int(input("Enter a number:"))

match op:
  case '+':
    print(num1+num2)
  case '-':
    print(num1-num2)
  case '*':
    print(num1*num2)
  case '/':
    if(num2==0):
      print("Cannot divide by zero")
    else:
      print(num1/num2)
```

```
Enter a number:2
Enter an operator:/
Enter a number:0
Cannot divide by zero
```

```
n = int(input("Enter a number:"))
prime = True
for i in range(2, int(n**0.5)+1):
  if (n%i==0):
    prime = False
    break
if(prime):
  print("Prime")
else:
  print("Not prime")
```

```
Enter a number:22
Not prime
```

```
import math
n = int(input("Enter a number:"))
sum = 0
num = n
while(n > 0):
  digit = n%10
  sum = sum +  math.pow(digit,3)
  n = n//10
if(sum == num):
  print("Armstrong")
else:
  print("Not Armstrong")
```

```
Enter a number:153
Armstrong
```

```
n = int(input("Enter a number:"))
sum = 0
num = n
while n>0:
  digit = n%10
  sum = sum*10 + digit
  n = n//10

if ( sum == num):
  print("Palindrome")
else:
  print("Not Palindrome")
```

```
Enter a number:1221
Palindrome
```

```
n = int(input("Enter a number:"))
fact = 1
for i in range(1, n+1):
  fact = fact * i
print(fact)
```

```
Enter a number:10
3628800
```

```
#fibonacci
n = int(input("Enter a number:"))
a = 0
b = 1
print(a, end = " ")
print(b, end = " ")
for i in range(2, n):
  c = a+b
  print(c, end = " ")
  a = b
  b = c
```

```
Enter a number:10
0 1 1 2 3 5 8 13 21 34
```

```
n = int(input("Enter a number:"))
for i in range(1, n):
  print("* "*i)
```

```
Enter a number:5
*
* *
* * *
* * * *
```

```
n = int(input("Enter a number:"))
for i in range(1,11):
  print(n , " * ", i, " = ", n*i)
```

```
Enter a number:17
17  *  1  =  17
17  *  2  =  34
17  *  3  =  51
17  *  4  =  68
17  *  5  =  85
17  *  6  =  102
17  *  7  =  119
17  *  8  =  136
17  *  9  =  153
17  *  10  =  170
```

String

```
word = input("Enter a word: ")
if ( word == word[::-1]):
  print("Palindrome")
else:
  print("Not Palindrome")
```

```
Enter a word: maverick
Not Palindrome
```

```
n = input("Enter a string:")
vowel=0
consonant=0
digit = 0
spaces = 0
special = 0
for i in n:
  if i.lower() in 'aeiou':
    vowel +=1
  elif i.isalpha():
    consonant +=1
  elif i.isdigit():
    digit +=1
  elif i.isspace():
    spaces +=1
  else:
    special +=1
print("Vowels:", vowel)
print("Consonants:", consonant)
print("Digits:", digit)
print("Spaces:", spaces)
print("Special characters:", special)
```

```
Enter a string:absdasdas!!!@@$#%#%$12345435   scac  ascasca 113213
Vowels: 7
Consonants: 13
Digits: 14
Spaces: 6
Special characters: 11
```

```
n = input("Enter a string:")
first5 = n[:5]
last5 = n[-5:]
rev = n[::-1]
sec = n[::2]
print(first5, last5, rev, sec)
```

```
Enter a string:MAVERICK
MAVER ERICK KCIREVAM MVRC
```

```
li = [1,2,0,3,4,5,6,8,7,2,3,4,5,6,9]
largest = li[0]
```

```
secondLargest = li[0]
smallest= li[0]
for i in range(1,len(li)):
  if li[i]>li[i-1]:
    if li[i]>largest:
      secondLargest = largest
      largest = li[i]
    if li[i]<smallest:
      smallest = li[i]
  elif li[i]<li[i-1]:
    if li[i]>secondLargest:
      secondLargest = li[i]
    if li[i]<smallest:
      smallest = li[i]
  else:
    if li[i]>secondLargest:
      secondLargest = li[i]
print("Largest:", largest)
print("Second Largest:", secondLargest)
print("Smallest:", smallest)
```

```
Largest: 9
Second Largest: 8
Smallest: 0
```

```
arr = [1,2,2,3,3,4,4,4,5,6,7,8,8,8,9,9,10]
res = []
for i in arr:
  if i not in res:
    res.append(i)
print(res)
print(set(arr))
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
arr = [1,2,3,4,5]
res=[]
for i in range(1,len(arr)):
  if arr[i] % 2 == 0:
    res.append(arr[i]*arr[i])
print(res)
```

```
[4, 16]
```

```
t_data = (10, 20, 30, 20, 40, 50, 20, 60)
print("Tuple:", t_data)

target = int(input("Enter element to find frequency: "))
frequency = t_data.count(target)
print(f"Frequency of {target}:", frequency)
```

```
print("Maximum value:", max(t_data))
print("Minimum value:", min(t_data))

Tuple: (10, 20, 30, 20, 40, 50, 20, 60)
Enter element to find frequency: 20
Frequency of 20: 3
Maximum value: 60
Minimum value: 10

students = {
    "Maverick": 90,
    "Akshat": 80,
    "Ayush": 60,
    "Saksham": 50
}
print("Initial dictionary:", students)


students["Mohit"] = 75
print("After adding Mohit:", students)

students["Saksham"] = 63
print("After updating Saksham's marks:", students)


if "Ayush" in students:
    del students["Ayush"]
print("After deleting Ayush:", students)

print("\nStudents scoring above 75:")
for name, marks in students.items():
    if marks > 75:
        print(f"{name}: {marks}")

Initial dictionary: {'Maverick': 90, 'Akshat': 80, 'Ayush': 60,
'Saksham': 50}
After adding Mohit: {'Maverick': 90, 'Akshat': 80, 'Ayush': 60,
'Saksham': 50, 'Mohit': 75}
After updating Saksham's marks: {'Maverick': 90, 'Akshat': 80,
'Ayush': 60, 'Saksham': 63, 'Mohit': 75}
After deleting Ayush: {'Maverick': 90, 'Akshat': 80, 'Saksham': 63,
'Mohit': 75}

Students scoring above 75:
Maverick: 90
Akshat: 80
```

SECTION-C

```
 def isPrime(num):
  prime=True
  for i in range(1, int(num**0.5)):
    if num%i==0:
      prime= False
      break
  return prime

print(isPrime(10))
print(isPrime(2))
print(isPrime(15))
```

```
False
True
False
```

```
arr = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x**2, arr))
print("Squares of all numbers:", squares)

even_squares = list(filter(lambda x: x % 2 == 0, squares))
print("Even squares:", even_squares)
```

```
Squares of all numbers: [1, 4, 9, 16, 25]
Even squares: [4, 16]
```

```
import functools

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

product = functools.reduce(lambda x, y: x * y, arr)
print("List:", arr)
print("Product of all numbers:", product)
```

```
List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Product of all numbers: 3628800
```

```
def calculate_sum(*args):
    total_sum = sum(args)
    return total_sum


def display_student_details(**kwargs):
    print("\n--- Student Details ---")
    for key, value in kwargs.items():
        print(f"{key.capitalize()}: {value}")


print("Sum of 5, 10, 15:", calculate_sum(5, 10, 15))
print("Sum of 1, 2, 3, 4, 5, 6, 7, 8, 9, 10:", calculate_sum(*range(1,
```

```
11)))

display_student_details(name="Maverick", roll_no=101, course="Computer
Science", grade="A")
```

```
Sum of 5, 10, 15: 30
Sum of 1, 2, 3, 4, 5, 6, 7, 8, 9, 10: 55

--- Student Details ---
Name: Maverick
Roll_no: 101
Course: Computer Science
Grade: A
```

```
file_name = "sample_file.txt"


with open(file_name, "w") as f:
    f.write("Line 1: Welcome to the world of Python programming.\n")
    f.write("Line 2: File handling is an important topic.\n")
    f.write("Line 3: We can read, write, and append data.\n")
    f.write("Line 4: This is the fourth line in our file.\n")
    f.write("Line 5: Learning is a continuous process.\n")

with open(file_name, "r") as f:
    content = f.read()


lines = content.strip().split("\n")
num_lines = len(lines)


words = content.split()
num_words = len(words)

print("--- File Content ---")
print(content)
print(f"Total Lines: {num_lines}")
print(f"Total Words: {num_words}")


additional_text = "\nLine 6: This line is newly appended to the file.\
n"
with open(file_name, "a") as f:
    f.write(additional_text)


with open(file_name, "r") as f:
    updated_content = f.read()
```

```python
print("\n--- Updated File Content ---")
print(updated_content)
```

```
--- File Content ---
Line 1: Welcome to the world of Python programming.
Line 2: File handling is an important topic.
Line 3: We can read, write, and append data.
Line 4: This is the fourth line in our file.
Line 5: Learning is a continuous process.

Total Lines: 5
Total Words: 43

--- Updated File Content ---
Line 1: Welcome to the world of Python programming.
Line 2: File handling is an important topic.
Line 3: We can read, write, and append data.
Line 4: This is the fourth line in our file.
Line 5: Learning is a continuous process.

Line 6: This line is newly appended to the file.
```

```python
try:
  n = int(input("Enter a number:"))
  print(10/n)
except ZeroDivisionError:
  print("Cannot divide by zero")
except ValueError:
  print("Invalid input")
finally:
  print("final")
```

```
Enter a number:a
Invalid input
final
```

```python
try:

    numerator = float(input("Enter the numerator: "))
    denominator = float(input("Enter the denominator: "))

    result = numerator / denominator

except ZeroDivisionError:
    print("Error: Cannot divide by zero!")

except ValueError:
    print("Error: Invalid input! Please enter numerical values.")

else:
```

```
    print(f"Success! The result of division is: {result}")

finally:

    print("Division operation attempt completed.")
```

```
Enter the numerator: 20
Enter the denominator: a
Error: Invalid input! Please enter numerical values.
Division operation attempt completed.
```

```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks

    def display_details(self):
        print("--- Student Details ---")
        print("Name:    ", self.name)
        print("Roll No: ", self.roll_no)
        print("Marks:   ", self.marks)

    def calculate_grade(self):
        if self.marks >= 90 and self.marks <= 100:
            print("Grade:    A")
        elif self.marks >= 80 and self.marks < 90:
            print("Grade:    B")
        elif self.marks >= 70 and self.marks < 80:
            print("Grade:    C")
        elif self.marks >= 60 and self.marks < 70:
            print("Grade:    D")
        else:
            print("Grade:    Fail")


stu = Student("Maverick", 17, 95)
stu.display_details()
stu.calculate_grade()
```

```
--- Student Details ---
Name:     Maverick
Roll No:  17
Marks:    95
Grade:    A
```

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

```
    def display_details(self):
        print(f"Employee Name: {self.name}")
        print(f"Salary: ${self.salary}")
        print("Role: General Employee")

class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department


    def display_details(self):
        print(f"Employee Name: {self.name}")
        print(f"Salary: ${self.salary}")
        print(f"Role: Manager of {self.department} Department")

class Developer(Employee):
    def __init__(self, name, salary, programming_language):
        super().__init__(name, salary)
        self.programming_language = programming_language

    def display_details(self):
        print(f"Employee Name: {self.name}")
        print(f"Salary: ${self.salary}")
        print(f"Role: Developer specialized in
{self.programming_language}")

print("--- Developer Details ---")
dev = Developer("Alice", 95000, "Python")
dev.display_details()

print("\n--- Manager Details ---")
man = Manager("Bob", 120000, "Engineering")
man.display_details()

print("\n--- Generic Employee Details ---")
emp = Employee("Charlie", 60000)
emp.display_details()
```

```
--- Developer Details ---
Employee Name: Alice
Salary: $95000
Role: Developer specialized in Python

--- Manager Details ---
Employee Name: Bob
Salary: $120000
Role: Manager of Engineering Department

--- Generic Employee Details ---
```

```
Employee Name: Charlie
Salary: $60000
Role: General Employee
```

```
import time


def measure_time(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        execution_time = end_time - start_time
        print(f"Execution time of '{func.__name__}':
{execution_time:.6f} seconds")
        return result
    return wrapper


@measure_time
def calculate_sum(n):
    total_sum = 0
    for i in range(1, n + 1):
        total_sum += i
    return total_sum


limit = 100000
result = calculate_sum(limit)
print(f"Sum of numbers from 1 to {limit:,}: {result}")
```

```
Execution time of 'calculate_sum': 0.014569 seconds
Sum of numbers from 1 to 100,000: 5000050000
```

```
def fibonacci_generator(limit):
    a, b = 0, 1
    while a <= limit:
        yield a
        a, b = b, a + b


limit_val = 100
print(f"Fibonacci numbers up to {limit_val}:")
for num in fibonacci_generator(limit_val):
    print(num, end=" ")
print()
```

```
Fibonacci numbers up to 100:
0 1 1 2 3 5 8 13 21 34 55 89
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

data = {
    'Student': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Math': [85, 92, 78, 89, 95],
    'Science': [90, 88, 85, 94, 91],
    'English': [78, 85, 82, 80, 88]
}
df = pd.DataFrame(data)
df.set_index('Student', inplace=True)

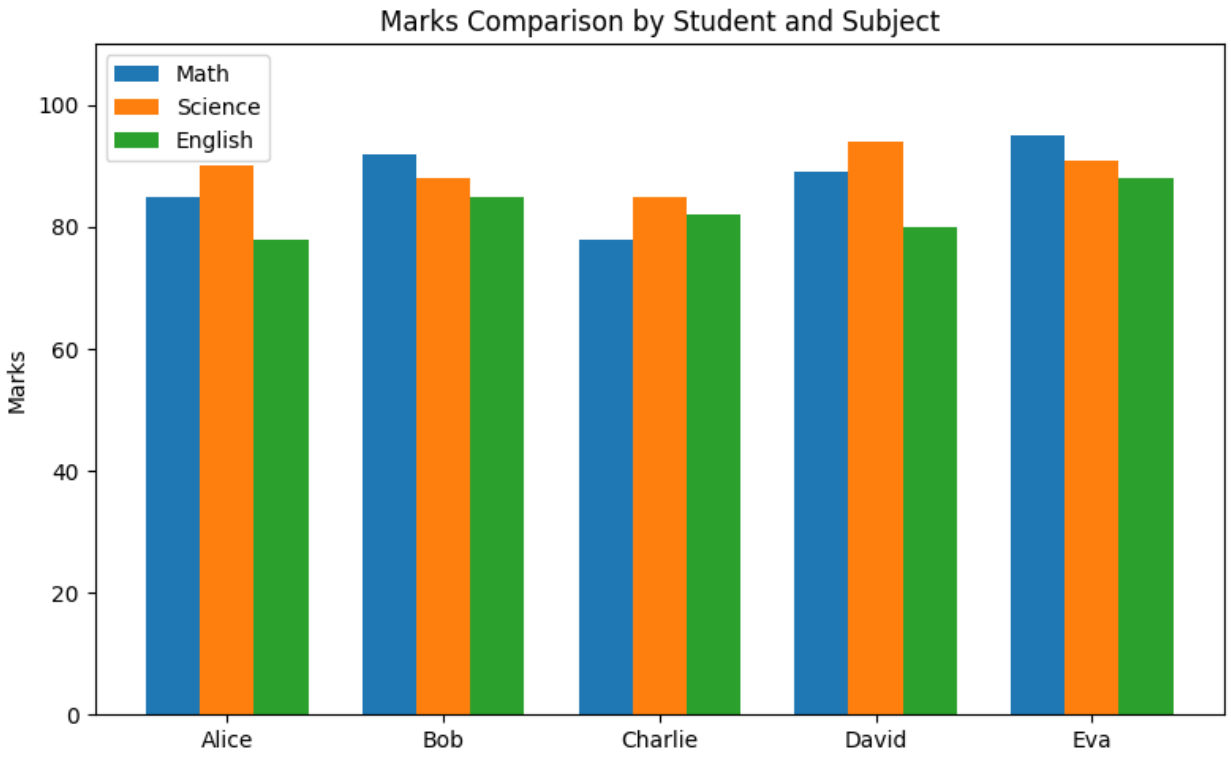

fig, ax = plt.subplots(figsize=(8, 5))
x = np.arange(len(df.index))
width = 0.25

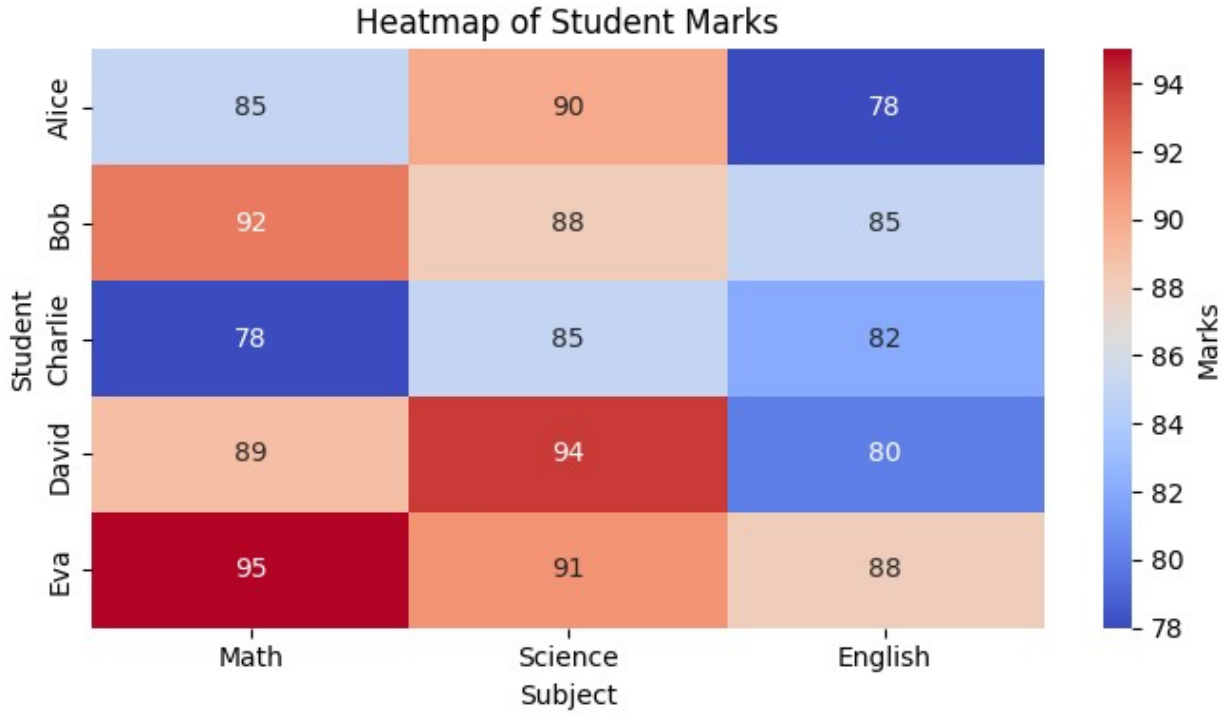

ax.bar(x - width, df['Math'], width, label='Math', color='#1f77b4')
ax.bar(x, df['Science'], width, label='Science', color='#ff7f0e')
ax.bar(x + width, df['English'], width, label='English',
color='#2ca02c')


ax.set_ylabel('Marks')
ax.set_title('Marks Comparison by Student and Subject')
ax.set_xticks(x)
ax.set_xticklabels(df.index)
ax.legend()
ax.set_ylim(0, 110)

plt.tight_layout()
plt.show()


plt.figure(figsize=(7, 4))
sns.heatmap(df, annot=True, cmap='coolwarm', fmt='d',
cbar_kws={'label': 'Marks'})
plt.title('Heatmap of Student Marks')
plt.ylabel('Student')
plt.xlabel('Subject')
plt.tight_layout()
plt.show()
```






```
import requests
from urllib.parse import urlencode
```

```python
url = "https://httpbin.org/get"
params = {'name': 'Maverick'}

try:
    print("--- Sending GET request using requests library ---")

    response = requests.get(url, params=params, timeout=5)


    print(f"Status Code: {response.status_code}")
    print("JSON Response:")
    print(response.json())

except requests.exceptions.Timeout:
    print("Error: The request timed out!")
except requests.exceptions.ConnectionError:
    print("Error: A connection error occurred!")
except requests.exceptions.RequestException as e:
    print(f"An error occurred: {e}")

print("\n--- Exploring urllib.parse.urlencode() ---")
query_params = {
    'name': 'Maverick Vinales',
    'course': 'Computer Science',
    'city': 'New York'
}

encoded_string = urlencode(query_params)
print("Original Dictionary:", query_params)
print("Encoded Query String:", encoded_string)
print("Full Constructed URL:", f"{url}?{encoded_string}")
```

```
--- Sending GET request using requests library ---
Status Code: 200
JSON Response:
{'args': {'name': 'Maverick'}, 'headers': {'Accept': '*/*', 'Accept-
Encoding': 'gzip, deflate, br, zstd', 'Host': 'httpbin.org', 'User-
Agent': 'python-requests/2.32.4', 'X-Amzn-Trace-Id': 'Root=1-6ace76da-
363be4ed395c1b5e438f05dc'}, 'origin': '34.91.156.23', 'url':
'https://httpbin.org/get?name=Maverick'}

--- Exploring urllib.parse.urlencode() ---
Original Dictionary: {'name': 'Maverick Vinales', 'course': 'Computer
Science', 'city': 'New York'}
Encoded Query String:
name=Maverick+Vinales&course=Computer+Science&city=New+York
Full Constructed URL: https://httpbin.org/get?
name=Maverick+Vinales&course=Computer+Science&city=New+York
```